HARDWARE MODULE / TECHNICAL DOCUMENTATION

[RTL module name]

[Module code]

Module revision	[Module revision / RTL commit]
Document version	0.1-draft
Date	November 2, 2026
Status	Draft; design data and verification results pending

Contents

1 About This Document **1**

1.1 Purpose 1

1.2 Scope 1

1.3 Identity and Revision History 1

1.4 How to Read This Document 1

2 Context and Requirements **2**

2.1 Problem and Use Cases 2

2.2 Module Boundary and Interfaces 2

2.3 Requirements and Acceptance Criteria 2

2.4 Assumptions and References 2

3 Module Architecture **3**

3.1 Functional Overview 3

3.2 Functional Blocks 3

3.3 Operating Modes and Failure Conditions 3

4 Engineering Reasoning and Design Decisions **4**

4.1 Alternatives Considered 4

4.2 Decision Record 4

4.3 Engineering Calculations and Analysis 4

4.4 Design Risks and Mitigations 4

5 RTL Implementation and Build Results **5**

5.1 RTL Structure and Behavior 5

5.2 Build Target and Configuration 5

5.3 Synthesis Results 5
5.4 Implementation Results 5
5.5 Bitstream Generation and Artifact Traceability 6

6 Verification and Evidence **7**
6.1 Verification Scope and Strategy 7
6.2 Test Plan and Pass Criteria 7
6.3 Execution Environment 7
6.4 Verification Figures 7
6.5 Results and Analysis 7
6.6 Requirements-to-Evidence Matrix 8
6.7 Evidence Artifact Types 8

7 Limitations and Future Work **9**
7.1 Known Limitations 9
7.2 Verification Gaps and Residual Risks 9
7.3 Planned Improvements 9

A Artifacts and Repository **10**
A.1 Repository and Module Folder 10
A.2 Artifact Index 10
A.3 Additional Supporting Material 10

CHAPTER 1

About This Document

1.1 Purpose

This document records the engineering work behind [RTL module name]: the problem being addressed, the reasons for the design choices, the implementation, verification evidence, and known limitations. Each section should be tied to the relevant module revision.

1.2 Scope

[*Describe the module's function, the subsystems covered, and what falls outside the scope of this document.*]

1.3 Identity and Revision History

Module	[RTL module name] ([Module code])
Module revision	[Module revision / RTL commit]
Document version	0.1-draft
Verification status	[*Not defined / in progress / complete; include date and revision*]

1.4 How to Read This Document

Requirements state the design targets; architecture and design decisions explain the approach; implementation records what was built; verification records the evidence; limitations identify what remains unresolved. The appendix links this document to its source artifacts.

CHAPTER 2

Context and Requirements

2.1 Problem and Use Cases

[*Describe the behavior or engineering problem addressed by this RTL module, its users, and the operating scenarios in scope.*]

2.2 Module Boundary and Interfaces

[*List ports, widths, direction, clock domains, reset polarity and behavior, handshake rules, and dependencies on the surrounding design. State what belongs outside this module.*]

2.3 Requirements and Acceptance Criteria

Give each requirement a stable ID so design decisions and tests can refer to it. Make acceptance criteria observable and measurable.

ID	Requirement / constraint	Acceptance criterion
REQ-01	[*Enter requirement*]	[*Metric, threshold, and test conditions*]

2.4 Assumptions and References

[*Record protocol assumptions, parameter values, target clock, reset expectations, external IP or interface specifications, and sources actually used. Add cited sources to the bibliography.*]

CHAPTER 3

Module Architecture

3.1 Functional Overview

[*Describe the datapath, control path, clock-domain boundaries, and end-to-end transfer of information.*]

3.2 Functional Blocks

[*For each block, record its purpose, inputs, outputs, dependencies, and related requirements. Refer to the source modules or RTL hierarchy.*]

3.3 Operating Modes and Failure Conditions

[*Describe reset, idle, active, stall/backpressure, completion, and error behavior. State how the module recovers from each supported failure condition.*]

CHAPTER 4

Engineering Reasoning and Design Decisions

4.1 Alternatives Considered

[*Describe architectures or control strategies that were actually considered. Compare trade-offs against the requirements and constraints.*]

4.2 Decision Record

Each significant decision should state its rationale, consequences, and traceable evidence.

ID	Related to	Decision and rationale
DEC-01	REQ-01	[*Choice, alternatives, rationale, consequences, and evidence source*]

4.3 Engineering Calculations and Analysis

[*Record equations, widths, signedness, latency, throughput, clock assumptions, margins, and the source of each input. Distinguish analytical estimates from simulation and implementation results.*]

4.4 Design Risks and Mitigations

[*Record failure modes, impact, basis for likelihood estimates, and mitigations.*]

CHAPTER 5

RTL Implementation and Build Results

5.1 RTL Structure and Behavior

[*Explain module hierarchy, datapath, FSM, registers, parameters, interfaces, clocking, and reset as implemented. Link the implementation to design decisions and source files.*]

5.2 Build Target and Configuration

[*Record top-level, defines and parameters, target FPGA and speed grade if applicable, tool version, constraints, target clock, and RTL commit. Distinguish a standalone RTL build from a top-level integrated build.*]

5.3 Synthesis Results

[*Record run status, optimizations or inferences, relevant warnings, and report paths. Results must match the target and configuration above.*]

Metric	Result	Report source / notes
LUTs / logic cells	[*Value*]	[*Path and run*]
Flip-flops / registers	[*Value*]	[*Path and run*]
BRAM / memory	[*Value*]	[*Path and run*]
DSP	[*Value*]	[*Path and run*]

5.4 Implementation Results

[*Record place-and-route status, final resource use, timing results, DRC summary, constraints, and every unmet rule or path.*]

Metric	Result	Report source / notes
Final utilization	[*Value*]	[*Path and run*]
Worst negative slack	[*Value, unit*]	[*Path and run*]
Total negative slack	[*Value, unit*]	[*Path and run*]
DRC / timing violations	[*Value*]	[*Path and run*]

5.5 Bitstream Generation and Artifact Traceability

[*If the module is integrated into an FPGA top-level, record bitstream status, file name, target device, source commit, and warnings. Otherwise state why bitstream generation is not applicable. A successful build does not establish functional verification.*]

CHAPTER 6

Verification and Evidence

6.1 Verification Scope and Strategy

[*State what is verified at module level, in top-level integration, and on hardware if applicable. Identify methods actually used: directed or randomized simulation, assertions, Python checks, notebooks, formal tools, or board tests. Keep functional verification distinct from build and timing results.*]

6.2 Test Plan and Pass Criteria

Define stimulus, expected response, execution conditions, and pass criteria before recording results. Every pass must point to a specific run.

Test	Requirement	Stimulus, check, and pass criterion	Status
VER-01	REQ-01	[*Case, oracle, result limit*]	Not run

6.3 Execution Environment

[*Record RTL and testbench commits, test top-level, parameters, simulator and version, Python dependencies, random seed, clock/reset, command, and run date. For notebooks, record kernel and execution order. For FPGA tests, identify the board and bitstream.*]

6.4 Verification Figures

Use the full-width layout for a complete multi-signal trace, the half-width layout for a focused waveform beside its explanation, or the fifth-width layout for a compact protocol marker. Keep every page portrait and use only the supported 1:1, 2:1, and 5:1 paper-width:image-width ratios.

6.5 Results and Analysis

[*For each VER-ID, state the actual result and pass/fail/not-run status, compare it with the pass criterion, explain deviations, and cite the artifact path and run ID. Include units for numerical results.*]

6.6 Requirements-to-Evidence Matrix

Requirement	Test	Evidence (repository-relative path, commit/run ID)
REQ-01	VER-01	[*Fill in after evidence and results are available*]

6.7 Evidence Artifact Types

Format	Role	Record
`.txt`	Log or tool output	Command, final status, and key result lines
`.vcd`	Signal transition trace	Signals, time range, and behavior demonstrated
`.py`	Stimulus, checker, or analysis	Invocation, dependencies, and result
`.ipynb`	Interactive experiment or analysis	Kernel, input data, cell order, and reproduced result

CHAPTER 7

Limitations and Future Work

7.1 Known Limitations

[*Describe performance limits, unsupported operating conditions, and their impact on users.*]

7.2 Verification Gaps and Residual Risks

[*Separate untested uncertainties from observed failures. State the reason and priority for follow-up.*]

7.3 Planned Improvements

[*Record proposed changes, affected requirements, and how they will be verified in the next revision.*]

CHAPTER A

Artifacts and Repository

A.1 Repository and Module Folder

Record the repository and fixed revision that contain the RTL and evidence for this document.

Reference	Value
Repository URL	[*Repository URL*]
Module folder in repo	[*Path relative to repository root*]
Reference commit or tag	[*Ref matching module revision and evidence*]
Direct folder link	[*URL to the module folder at that ref*]

Use a fixed ref so readers can find the RTL, build reports, and verification evidence that match this document version.

A.2 Artifact Index

Type	Location / revision	Description
RTL and constraints (if any)	[*Path and commit*]	[*Enter details*]
Testbench / test script	[*Path and commit*]	[*Enter details*]
Synthesis report	[*Path and run*]	[*Enter details*]
Implementation report	[*Path and run*]	[*Enter details*]
Bitstream (if any)	[*Path and hash*]	[*Enter details*]
Verification evidence	[*Path and run*]	[*.txt, .vcd, .py, .ipynb; as present*]

A.3 Additional Supporting Material

[*Place waveforms, long tables, build configurations, simulation results, or detailed procedures here when they are needed as evidence.*]